

Principles

These principles exist to keep Overdex focused as the project grows.

When two solutions are possible, the one that best follows these principles should be preferred.

Build First

Ideas are valuable.

Working software is more valuable.

Prototype quickly, learn from the result, then improve it.

Show That It Happened

Do not assume a feature works.

Demonstrate it.

A successful build, visible behavior, screenshot, log, or test is worth more than confidence.

Evidence beats assumptions.

One Feature. One Commit. One Test.

Large changes hide bugs.

Small changes reveal them.

Complete one feature.

Verify it.

Commit it.

Move on.

Build Foundations Before Features

A stable foundation creates many future features.

Do not rush visible functionality if the underlying architecture is not ready.

The Pokédex came before the overlay because knowledge comes before advice.

Observe Before Inferring

The battle engine should distinguish between what it knows and what it believes.

Observed information is evidence.

Inferred information is a conclusion built from evidence.

The two should never be confused.

The Overlay Speaks Last

The overlay does not think.

It does not guess.

It presents conclusions reached by the systems beneath it.

Its job is clarity, not computation.

Every Pixel Must Earn Its Place

Screen space is limited.

Attention is even more limited.

If an element does not improve a decision, it should not exist.

The overlay should never compete with the game.

It should quietly complement it.

Recognition Over Reading

Recognition is faster than reading.

Prefer icons, color, position, and shape over text whenever practical.

Text is a last resort.

The player should understand the overlay with a glance, not a sentence.

Minimalist. Game Boy. Winning.

Avoid decoration.

Prefer purpose.

Interfaces should feel deliberate rather than flashy.

Simple interfaces often communicate more effectively than complex ones.

Trust Through Consistency

Every icon should have one meaning.

Every color should have one purpose.

Every interaction should behave the same way every time.

Consistency builds confidence.

Preserve the Player's Attention

The player is here to battle.

Not to operate another application.

Overdex should reduce cognitive load, not increase it.

The best interaction is often the one the player never notices.

Motion Must Earn Attention

Motion is the strongest visual signal available.

Use it sparingly.

If an element moves, the player should immediately understand why.

Calm interfaces build trust.

Design for Learning

The best feature is not one that tells the player what to do.

It is one that teaches them something they will remember.

If Overdex helps a player make better decisions without Overdex tomorrow, it has succeeded.

Respect Uncertainty

Not every conclusion deserves complete confidence.

When the application is uncertain, it should communicate uncertainty honestly rather than pretending certainty.

Trust is difficult to earn and easy to lose.

The Project Is Never Finished

Overdex is expected to evolve.

Architectures will improve.

Interfaces will change.

Ideas will mature.

The goal is not perfection.

The goal is to make the next version better than the last.

Above All

Make the player's experience feel effortless.

If a feature makes the player think about Overdex instead of Pokémon, redesign the feature. Red is debt.

Green is proof.

Zero red before starting the next feature.

Every commit should leave the project in a state you'd be happy to hand to another developer.

Promote data from transient to permanent only once

Trust the Data

Overdex never presents certainty where only confidence exists.

Every observation contributes evidence.

Every recommendation reflects confidence.

Multiple independent observations are preferred over a single source.

Show, Don't Assume

A feature is not complete because it compiles.

Features are complete when they have been:

Built

Run

Verified

Observed

ORBSLOP

The player should learn Overdex's controls once.

Every screen should reuse the same interaction model whenever practical.

Consistency builds speed.

Familiarity builds confidence.

The D-pad owns focus.

Meaning:

In lists, it moves the current selection. In editors, it manipulates the current selection. In menus, it changes the highlighted option. In brackets, it moves through matches. In settings, it changes the active setting.

The D-pad always answers the same question:

"Where is my attention?"

What changes is what "moving" means in that context.

Respect Existing Architecture

Assume the existing architecture is intentional.

Do not improve it unless the task explicitly asks you to.

Working code is not a problem to solve.

Prefer extending existing behavior over replacing it.

The smallest successful change is usually the correct change.

Refactors are features. Treat them with the same planning, testing, and review as any other feature.

Confidence Over Certainty

Overdex never presents uncertain information as fact.

When confidence is high, Overdex speaks clearly.

When confidence is reduced, Overdex communicates why.

Users should always understand not only what Overdex believes, but how strongly it believes it.

Trust Is Earned

Every interaction should increase or preserve user trust.

Overdex would rather admit uncertainty than provide false confidence.

A feature that is 90% correct should communicate that it is 90% correct.

Overdex is a Pokémon companion that earns trust by making its confidence visible.

Navigation Principle

Gestures are not used for navigation within the Overdex device.

The sole exception is the native Android system gesture used to leave the application entirely.

This gesture represents returning from the Overdex handheld to the user's phone, not navigating within the handheld itself.